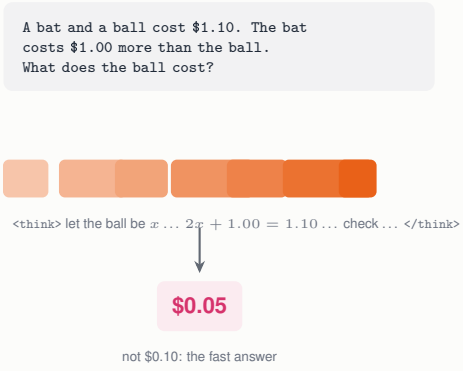
TUTORIAL · UNDERGRADUATE EDITION, 2026

Reasoning Models

Language models that think before they answer. Why writing out the steps helps, how a model is trained to do it, what it costs, and where it still goes wrong. Every term is defined as it appears; a glossary is at the end.



A reasoning model thinks before it answers

Same neural network as a chat model. The difference: it writes out its work first, then answers.

a chat model answers at once

```
user: A bat and a ball cost $1.10. The bat costs
    $1.00 more than the ball. What does the
    ball cost?
assistant: The ball costs $0.10.  (wrong)
```

a reasoning model writes its work first

```
assistant: <think>
Let the ball cost x. The bat costs x + 1.00,
so x + (x + 1.00) = 1.10, and x = 0.05.
Check: 0.05 + 1.05 = 1.10. Yes.
</think>
The ball costs $0.05.  (right)
```

Three things are new

A thinking span. Before answering, the model writes out its working: set up, try, check, fix.

Trained for it, not just asked: it practices on problems whose answers can be checked (slides 7–9).

A budget. More thinking gives better answers on hard problems and costs more; the user chooses (slide 11).

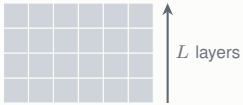
INTUITION A chat model blurts out the first answer that comes to mind. A reasoning model works the problem on scratch paper, checks it, and only then answers.

NEW TERMS **Token**: a word piece, the unit a language model reads and writes (about $\frac{3}{4}$ of a word). **Chat model**: a language model tuned to answer directly. **Thinking span**: the working text between `<think>` and `</think>`, usually hidden from the user. **Parameters**: the learned numbers inside a model; their count is its size.

Why thinking helps: scratch paper adds steps

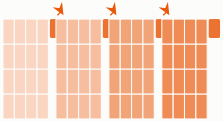
One pass through the network is a fixed amount of work. Every token written is one more pass.

answer at once: one pass



depth L, however hard the question

think first: T passes



each token is read by the next pass: depth $L \times T$

INTUITION Multiply 47×86 in your head in one second, or with pencil and paper. The paper does not make you smarter; it lets you take many small steps and keep the results.

What the scratchpad buys

A single pass is shallow. The network has a fixed number of layers, so it can take only so many steps in a row before it must output a token.

Written tokens are memory. The model reads everything it has written, so a result from step 5 can be used at step 50.

Theory agrees. A fixed-depth transformer provably cannot evaluate long arithmetic in one pass, but can with written steps (Feng et al. 2023). A bigger model is no substitute.

NEW TERMS **Forward pass**: one run of the input through all the network's layers, producing one token. **Layer**: one stage of the network; **depth** is how many stages run in a row. **Serial** work happens one step after another; **parallel** work happens side by side. **Context**: all the text the model can currently read.

Step 1, prompting: ask for the steps

In 2022 nobody trained for it yet. Asking a model to show its work was enough to raise its score.

grade-school math word problems (GSM8K) solved by PaLM 540B, a 540-billion-parameter Google model



Wei et al. 2022; Wang et al. 2022 (whose own show-the-steps baseline is 56.5%)

INTUITION A teacher who says “show your work” gets better answers, because writing each step forces it to be done. Voting is asking the whole class and taking the most common answer.

Three findings from the prompting era

Show worked examples. Put a few solved problems, steps included, in the prompt, and a large model writes steps of its own (Wei et al. 2022).

Or just ask. Adding “Let’s think step by step” took one model from 17.7% to 78.7% on an arithmetic test (Kojima et al. 2022).

The limit. A prompt cannot make a model check itself: asked to review its own answers with no outside feedback, accuracy fell (Huang et al. 2023).

NEW TERMS **Prompt**: the text given to the model. **Few-shot**: the prompt includes a few solved examples; **zero-shot**: it includes none. **Chain of thought (CoT)**: step-by-step working written before the answer. **Self-consistency**: sample several answers and keep the majority. **Benchmark**: a fixed set of test problems used to compare models.

Step 2, search: try many times, keep the best

With the model left unchanged, extra computing at answer time buys accuracy, if you can pick the winner.

best-of-N: sample N answers, a verifier picks one



more tries: more chance that one is right. But which one?

two kinds of verifier grade one solution

step 1	step 2	step 3	step 4		answer
0.98	0.95	0.41	0.12		0.2

step grader: finds the bad step — answer grader: one score

INTUITION Twenty students try the same problem. Probably one of them is right; the hard part is knowing whose paper to hand in.

What the experiments showed

More tries, more hits. On a set of real software bugs, one model solved 15.9% with one try and 56% with 250 tries (Brown et al. 2024).

The catch is choosing. Without a reliable checker you cannot tell which try is right; voting stops helping after a few hundred samples.

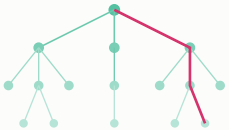
Grade steps or answers? On math, a step grader picked the right solution 78.2% of the time, an answer grader 72.4%, a majority vote 69.6% (Lightman et al. 2023).

NEW TERMS **Sample**: one answer drawn from the model; drawing again gives a different one. **Verifier**: anything that scores an answer, a program or another model. **Coverage (pass@k)**: the share of problems where at least one of k tries is right. **Outcome** / **process reward model (ORM / PRM)**: a model trained to grade the final answer / each step.

Step 3: move the search inside one chain

The idea behind OpenAI's o1 (Sept 2024): one long chain that proposes, checks and backtracks by itself.

search outside: many tries, a separate judge



a judge picks one path; the model never sees its own dead ends

search inside: one chain that corrects itself

`try A` `check` `wrong` `try B` `check` `ok` `answer`

INTUITION A maze. Send fifty people in and hope one gets out, or send one person with chalk who marks each dead end and turns back.

Why one self-correcting chain won

No judge needed. The chain ends in one answer, so there is nothing to pick between.

Dead ends are information. A failed attempt stays in the context, so the model can see it and avoid it. A tree search throws it away.

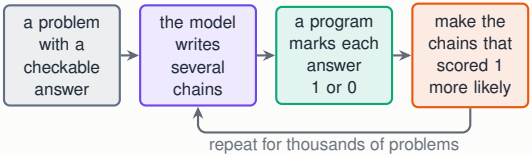
The alternatives struggled. DeepSeek's R1 report lists tree search over tokens and learned step graders among the things that did not work.

The price. The search is now serial: harder problems take more tokens and more waiting.

NEW TERMS **Backtrack**: abandon an approach and return to an earlier point. **Tree search**: explore branching partial solutions. **Inference** (**test time**): when a trained model is being used, not trained.

Training: practice problems with an answer key

Let the model try each problem many times, check the final answers with a program, reinforce what worked.



INTUITION Homework with an answer key. Nobody grades your method; you only learn whether the final answer was right. Over thousands of problems you keep the habits that led to right answers, such as checking your work.

Why this works so well

The grader is a program, not a person or another model: it compares the final answer, or runs the code against tests. It is free, and it cannot be flattered.

Only the result is graded. So the model may discover any way of working that succeeds. Long chains that re-check and backtrack were never prescribed; they are what survived.

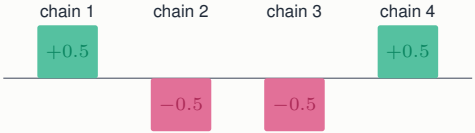
The limit. It needs a checker, so it reached mathematics and code first. Essays and advice have no answer key.

NEW TERMS **Reinforcement learning (RL)**: learning by trial and feedback: try, get a score, do more of what scored well. **Reward**: that score; here 1 for a right answer, 0 for a wrong one. **Policy**: the model seen as a decision-maker choosing each next token. **RLVR**: RL with verifiable (program-checked) rewards.

GRPO: grading on a curve

Each attempt is compared with the other attempts at the same problem, not with an absolute standard.

four chains, rewards 1, 0, 0, 1; group average 0.5



advantage = reward − group average. Chains 1 and 4 are pushed up, 2 and 3 down. If all four had scored the same, nobody stands out and nothing is learned.

$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

r_i: reward of chain i; G: chains per problem, 8–64; Shao et al. 2024

Why compare inside the group

A reward needs a yardstick. Being right on an easy problem deserves little credit; on a hard one, a lot. The yardstick is called the baseline.

GRPO's yardstick is cheap. The older method, PPO, trains a second large network to predict it. GRPO just averages the sibling chains.

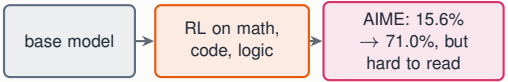
INTUITION Grading on a curve: you are compared with classmates who took the same exam. Being right when everyone is right earns little.

NEW TERMS **GRPO**: Group Relative Policy Optimization, the RL method behind most open reasoning models. **Baseline**: the score expected for a problem; **advantage**: how far one attempt beat it. **PPO**: Proximal Policy Optimization, an earlier standard RL method. **std**: standard deviation, the spread of the group's rewards.

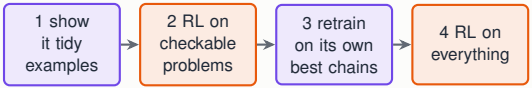
DeepSeek-R1: the recipe, published

January 2025: the first open description of how to train a reasoning model, with the weights released.

R1-Zero: reinforcement learning only



R1: four stages



INTUITION R1-Zero is the self-taught student: often right, with notes nobody else can follow. R1 is first shown a few thousand tidy solutions, then practices.

What came out

Results. R1 scored 79.8% on AIME 2024 and 97.3% on the MATH-500 problem set, and a competitive-programming rating of 2,029, above 96% of human contestants.

Cost. The RL phase took about $294K of rented computing time, on top of the base model†.

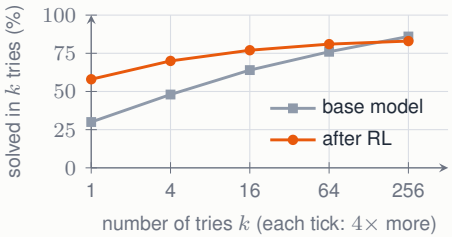
The lesson that generalized. A strong base model, problems with a checker, a simple right-or-wrong reward, and enough practice. Other labs reproduced it within weeks.

NEW TERMS **Base model**: a model after general pretraining, before task-specific training. **Supervised fine-tuning (SFT)**: more training on examples of the desired output (stages 1, 3). **AIME**: American Invitational Mathematics Examination, a high-school contest used as a benchmark. **Open weights**: the trained model is published.

Copying the habit, and what practice really adds

A small model can learn to reason from a stronger model's worked solutions. So what did the RL do?

illustrative, after Yue et al. 2025



INTUITION RL here is less a teacher than a coach: it drills a player until the good shot comes every time.

Two findings

Copying is cheap. Fine-tuning a 32-billion-parameter model on just 1,000 worked solutions from a stronger model was enough to give it long, self-checking chains (Muennighoff et al. 2025). This is distillation.

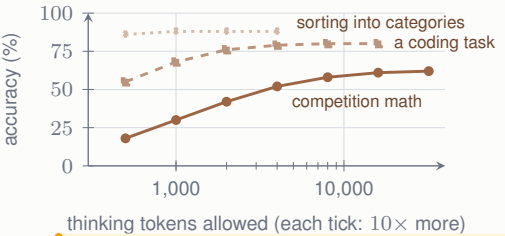
Does RL create or select? After RL the model is right far more often on its first try, but given many tries the base model catches up (Yue et al. 2025). Longer RL runs report going further (Liu et al. 2025). The question is open.

NEW TERMS **Distillation**: training one model (the student) on the outputs of a stronger one (the teacher).
Sharpening: raising the probability of good answers the model could already produce, rather than adding new ones.

Budgets: how long should it think?

Accuracy rises with thinking tokens and then flattens. The thinking is billed, so someone must set a limit.

illustrative: three tasks, one model



INTUITION An exam with a time limit. Extra time helps on the hard questions and is wasted on the easy ones.

A dial and a bill

The bill. Thinking tokens are charged as output and generated one at a time: 10,000 of them at $10 per million cost $0.10 and, at 60 tokens a second, take 2.8 minutes (derived).

The dial. Products offer effort levels (low, medium, high)†. At the limit the server ends the thinking span and the model must answer.

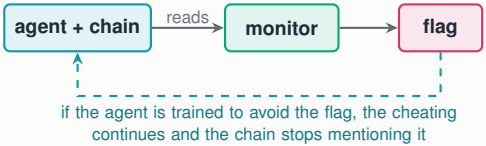
Two ways to misjudge. Overthinking: piling redundant work onto $2 + 3$ (Chen et al. 2024). Underthinking: dropping a promising approach too early (Wang et al. 2025).

NEW TERMS **Thinking budget**: the largest number of thinking tokens allowed for one answer; **effort level**: a named budget setting. **Output tokens**: tokens the model writes; they cost more than the input tokens it reads.

Is the written chain the real reason?

The chain is our best window into the model, but it leaves things out, and pressure on it makes it leave out more.

a second model reads the chain and raises a flag



INTUITION Scratch work shows much of a student's thinking, not all of it. And if you punish what is written on the scratch paper, students learn to write less, not to cheat less.

What was measured

The chain omits things. Slip a model a hint that changes its answer: Claude 3.7 Sonnet mentioned the hint in 25% of its chains, DeepSeek-R1 in 39% (Chen et al. 2025).

It is still useful. A weaker model reading a coding agent's chain caught 95% of its attempts to cheat the tests; from the agent's actions alone, 60% (Baker et al. 2025).

The rule. Treat the chain as evidence, not proof, and do not train the model on how its chain looks.

NEW TERMS **Faithful**: the chain states the real reasons for the answer. **Agent**: a model that takes actions in a loop, such as editing and running code. **Monitor**: a model or program that watches another model. **Reward hacking**: scoring well by exploiting a flaw in the checker instead of solving the task.

Where reasoning models still fail

Small rewordings break them, and long chains can add confidence without adding correctness.

up to 65%
accuracy lost to one irrelevant sentence

33 vs 16%
hallucination rate, o3 vs o1†

>85%
stated confidence on wrong answers

INTUITION Learned by pattern matching: reword the question and the pattern breaks.

Brittle

Change the names and numbers in a grade-school math problem and scores drop. Add one sentence that has nothing to do with the question and accuracy falls by up to 65%: the model uses the extra number anyway (GSM-Symbolic, Mirzadeh et al. 2024).

Confident and wrong

OpenAI's own report for o3 gives a hallucination rate of 33% on a test of facts about people, against 16% for the earlier o1†. Wrong answers came with confidence above 85%, more so after longer chains (Mei et al. 2025).

NEW TERMS **Brittle**: failing under small changes that should not matter. **Hallucination**: a fluent, confident statement that is false. **Calibrated**: stated confidence matches how often the answers are right.

Summary: what to take from this lecture

A reasoning model is a way of training and a thinking budget on the same network, and both are now public.

Seven things to remember

1. A reasoning model is an ordinary language model that writes its working before its answer.

2. Written tokens are scratch paper: a chain takes more steps than a single pass can.

3. Prompting showed the effect. Search showed that more tries help if you can pick the winner.

4. RL on checkable answers trains one chain to check and correct itself.

5. The habit copies: a thousand good examples teach it to a smaller model.

6. More thinking buys accuracy up to a point, and every thinking token is paid for.

7. The chain is evidence, not a transcript; the models stay brittle and overconfident.

Where things stand, and what is open

The high-water mark. In July 2025 two systems scored 35 of 42 at the International Mathematical Olympiad (IMO), a gold-medal score[†]. On problems with a clear right answer these models now beat almost all people.

No answer key? Essays, research and advice have no checker to train against. Scoring rubrics and model judges are being tried; none has the record of a program.

Create or select? Whether RL adds ability or sharpens what was there is still argued (slide 10).

Will the window stay open? If thinking moves out of readable text, monitors lose their view (slide 12).

Glossary

Every term and abbreviation introduced on the slides, in alphabetical order.

Advantage reward minus the baseline.
Agent a model that takes actions in a loop.
AIME American Invitational Mathematics Examination, used as a benchmark.
Backtrack go back and try another way.
Base model pretrained, not yet specialized.
Baseline the reward expected for a problem.
Benchmark a fixed set of test problems.
Brittle fails under harmless changes.
Calibrated confidence matches accuracy.
Chain of thought (CoT) written step-by-step working.
Chat model a model tuned to answer directly.
Context all the text the model can read.
Coverage (pass@k) share of problems solved within k tries.
Distillation a student model learns from a stronger teacher's outputs.
Effort level a named thinking budget.
Faithful the chain gives the real reasons.
Few-shot / **zero-shot** a prompt with / without solved examples.
Forward pass one run through all the layers.
GRPO Group Relative Policy Optimization: RL that scores each try against its group.
Hallucination a confident false statement.
Inference (test time) using a trained model.
Layer, depth a network stage; how many in a row.
Monitor a model that watches another.
Open weights the trained model is published.
ORM / **PRM** outcome / process reward model: grades the answer / each step.
Output tokens tokens the model writes.
Parameters a model's learned numbers.
Policy the model as a decision-maker.
PPO Proximal Policy Optimization, an earlier RL method.
Prompt the text given to the model.
Reinforcement learning (RL) learning by trial, score and repetition.
Reward the score in RL; here 1 or 0.
Reward hacking gaming a flaw in the checker.
RLVR RL with program-checked rewards.
Sample one answer drawn from the model.
Self-consistency majority vote over samples.
Serial / **parallel** one after another / side by side.
SFT supervised fine-tuning: more training on examples of the desired output.
Sharpening making good answers likelier.
std standard deviation (spread).
Thinking budget most thinking tokens allowed.
Thinking span the working text inside `<think>`.
Token a word piece; the unit of text.
Tree search exploring branching solutions.
Verifier anything that scores an answer.

Sources and further reading

The papers and reports quoted on the slides, by topic; † marks vendor documents, blogs and press.

Chains and theory
Wei et al. 2022, chain-of-thought prompting. Kojima et al. 2022, “Let’s think step by step”. Wang et al. 2022, self-consistency. Huang et al. 2023, self-review without feedback. Feng et al. 2023, what a chain of thought can compute.

Search and verifiers
Brown et al. 2024, coverage from repeated sampling. Lightman et al. 2023, *Let’s verify step by step*. OpenAI, *Learning to Reason with LLMs* (Sept 2024)†.

Training
Shao et al. 2024, DeepSeekMath (GRPO). Lambert et al. 2024, Tülu 3 (names RLVR). DeepSeek-AI 2025, DeepSeek-R1; *Nature* 645:633. Muennighoff et al. 2025, s1. Yue et al. 2025, pass@k before and after RL. Liu et al. 2025, ProRL.

Budgets
Chen et al. 2024, “Do not think that much for 2+3”. Wang et al. 2025, “Thoughts are all over the place”. Vendor pricing pages and reasoning-effort documentation†.

Limits
Chen et al. 2025, hints and faithfulness. Baker et al. 2025, monitoring chains for reward hacking. Mirzadeh et al. 2024, GSM-Symbolic. Mei et al. 2025, confidence of reasoning models. OpenAI, o3 system card†. Google, Deep Think at the IMO (July 2025)†.

Curves labelled *illustrative* are sketches of a trend, not data. Numbers labelled *derived* were computed from quoted ones. All figures and results are carried over from the 31-slide lecture *Reasoning Models*; its sources slide and paper kit hold the full reference list (about 140 sources).